



HashCloak

Code Review and Security Assessment
For
Privacy Cash Core EVM

Version 5.0: July 24th, 2026

Prepared For:

Privacy Cash

Prepared by:

Manish Kumar | *HashCloak Inc.*

Table Of Contents

Executive Summary	2
Version Info	4
Scope	4
Overview	5
Methodology	5
Overview of Evaluated Components	6
Findings	7
PCM-1: No Emergency Pause mechanism in EtherPool contract	7
PCM-2: Dead code: unused Account struct and PublicKey event	8
PCM-3: Violates the Checks-Effects-Interactions Pattern in transact	9
PCM-4: Custom implementation of nonReentrant modifier instead of using OpenZeppelin's reentrancy guard contract	9
PCM-5: Floating Solidity pragma directives are used	10
PCM-6: Missing check to ensure that _minimumAmount is less than _maximumDepositAmount	10
References	11
Appendix	11

Executive Summary

Privacy Cash engaged *HashCloak Inc.* to perform a code review and security assessment for Privacy Cash Core EVM, an EVM version of Privacy Cash inspired from Tornado Cash, supporting private ETH transfers on Base. The protocol was then later extended to support on Ethereum mainnet, BNB Chain and later to Robinhood chain.

During the audit period, we familiarized ourselves with the overall concepts of Tornado Cash and Privacy Cash and its core components relevant to the audit scope by reviewing the code and the provided documentation. We also assessed some security vulnerabilities commonly found in Solidity and Circom projects. Subsequently, we examined the code for errors, primarily focusing on logical correctness and identifying potential issues that could lead to double spending, loss of funds, or other privacy-related issues. We also ran slither static analyzer tools to find common vulnerabilities in Solidity contracts. Since we earlier did the audit of Circom circuits for Privacy Cash and the same circuit was used in the EVM version with the verifier generated using Snarkjs. Therefore the only necessary check was that the verification key is correct and is the same as that of used in the Privacy Cash. The solidity contracts were found to be adapted from Tornado Cash.

Overall, we found the source code is of good quality in representing the entities in the specification. The code is modular and well-structured.

Later, the team added the USDC Support in [PR #9](#) which was reviewed and subsequently merged in commit [1e9af6a33d1bd2110d9260a413cf1953490128cc](#). This implementation introduces a single privacy pool per ERC-20 token i.e., one contract deployment per token.

On May 19th, the evm code was deployed on Ethereum mainnet supporting the USDT tokens privacy pool transfer. The deploying script was modified to support Ethereum mainnet native USDT address. The deployed contract was checked for the correct USDT contract and token addresses. We found all the deployed addresses to be correct with the proxy contract correctly pointing to the ERC Pool contract and the constructor arguments of ERC Pool also matching with the verifier, hasher and token contracts. All the changes are reflected in the commit: [8d524af380af69c2fbeb63106ce0adf12aaa2d13](#)

Later, on July 13, 2026, two PRs ([#11 "Support BNB Chain"](#) and [#12 "deploy BNB USDT pool"](#)) was introduced that extends Privacy Cash EVM protocol to the BNB smart chain. The deployment script and config was modified to support native BNB and BSC-USD tokens. The deployed contract was checked for the correct BNB and BSC-USD pool contract and token addresses. We found all the deployed addresses to be correct with the proxy contract correctly pointing to the EtherPool's and ERCPool's contracts. We did not find any vulnerabilities or any hardcoded secrets in the mentioned PRs. The changes for the BNB pool were merged in commit [5d795b9957cae2ce90223b9478f0b0a0b51becca](#).

On July 24, 2026, Robinhood Ether pool PR([PR #13](#)) was introduced which extends the Privacy Cash EVM protocol to the Robinhood Chain. The PR adds a robinhood network entry to `hardhat.config.js` and Blockscout-based contract verification config. We found all the on-chain deployed addresses to be correct with the proxy contract correctly pointing to the ERC Pool contract. We did not find any vulnerabilities or any or any hardcoded secrets in the mentioned PR. The changes were merged in commit [20f64fcb40069ceb5a047beb6285d0e300cbc717](#).

We found issues ranging from low to Informational:

Severity	Number of Findings	Status
Critical	0	N/A
High	0	N/A
Medium	0	N/A
Low	1	PCM-6 fixed
Informational	5	PCM-1, PCM-2, PCM-4 and PCM-5 fixed, PCM-3 acknowledged

Version Info

Versions	Delivery Date
Final Delivery	March 17th, 2026
Final Version 2.0	April 6th, 2026
Version 3.0	May 21st, 2026
Version 4.0	July 13th, 2026
Version 5.0	July 24th, 2026

Scope

For the audit, we considered the following repositories:

- <https://github.com/SocialfiPanda/privacy-cash-core-evm> at commit 97974f5f6a2f0b5142b32e5d9f6b014db9912399

With following files in scope:

- privacy-cash-core-evm
 - contracts/*
 - circuits/*
 - scripts/deployTestnet.js

The upgrade script was updated later and was included in the review as shown below:

- <https://github.com/SocialfiPanda/privacy-cash-core-evm> at commit a2eb919cbd6358b21f09786549aa65e451b0a412

With following files in scope:

- scripts/[deployUpgrade.js](#)

Later The following PRs were also provided for a review:

- <https://github.com/SocialfiPanda/privacy-cash-core-evm/pull/7>
- <https://github.com/SocialfiPanda/privacy-cash-core-evm/pull/8>

In the second version, the following PR was included for review:

- <https://github.com/SocialfiPanda/privacy-cash-core-evm/pull/9/>

Overview

Privacy Cash Core EVM is an EVM version of Privacy Cash inspired from Tornado Cash, supporting private ETH transfers on Base. The protocol implements a privacy-preserving transaction system, drawing inspiration from Tornado Cash but adapted for the Base environment. It allows users to privately deposit and withdraw funds by leveraging zero-knowledge proofs (specifically Groth16) and a Merkle tree structure. Each deposit generates a unique commitment that is inserted into a Merkle tree maintained on-chain. Withdrawals are performed by proving knowledge of a valid note inside the tree using a zero-knowledge proof and corresponding nullifier to prevent double-spending.

Methodology

The audit was conducted through a combination of manual verification and using static analysis tools and cross verifying the vulnerabilities. The primary focus was to map the available specifications and existing resources such as Tornado Nova and Privacy Cash to the actual implementation and also using different static analyzers for Solidity contracts such as Slither. We also assessed the code for sufficient code coverage and various types of vulnerabilities, including:

- Authorization checks while transact
- Nullifier checks for double spending
- Merkle root validation

- Fee Logic and Transfers
- Underconstrained circuits
- Look for any dead code
- Any attack that can impact user funds
- Any undefined behavior
- Any transaction replay attack or double spending possibility
- Validation and access control of the admin
- Reentrancy attacks
- Any uninitialized state or storage variables
- Unchecked-transfer

Overview of Evaluated Components

circuits

- Compared with previously audited Privacy Cash Circuits

Contracts

- Authorization checks while transact
- Nullifier checks for double spending
- Merkle root validation
- Fee Logic and Transfers
- Account ownership
- Look for any dead code
- Any attack that can impact user funds
- Any undefined behavior
- Transaction replay
- Validation and access control of the admin
- Validation of verification keys

Findings

PCM-1: No Emergency Pause mechanism in **EtherPool** contract

Type: Informational

Files affected:

- contracts/EtherPool.sol

Description: The `transact()` function of **EtherPool** contract controls all value movement in the protocol, including handling deposits, withdrawals, and shielded transfers with ZK proof verification. However, the contract does not include any pause mechanism allowing the admin to disable operations in an emergency situation. Without such a mechanism if any issue is identified in production, the protocol cannot be stopped immediately. The only change which can be done in the current system is changing the maximum deposit limit to 0 thus not allowing any new deposit but this is not sufficient and attackers can still exploit the system if they have already deposited any funds.

Impact: If a vulnerability is discovered, the protocol cannot immediately halt operations.

Recommendation: Add Pausable mechanism to allow the admin to pause the operation in an emergency.

Status: The team added the Pausable mechanism in this PR:

<https://github.com/SocialfiPanda/privacy-cash-core-evm/pull/6>

PCM-2: Dead code: unused **Account** struct and **PublicKey** event

Type: Informational

Files affected:

- contracts/EtherPool.sol

Description: In contract **EtherPool**,

```
struct Account {  
    address owner;  
    bytes publicKey;  
}
```

and,

```
event PublicKey(address indexed owner, bytes key);
```

are defined but not used anywhere in the codebase. If there is no intention to use it then it must be removed from the codebase.

Recommendation: Remove **Account** struct and **PublicKey** event from the codebase.

Status: The team provided a fix in this PR:

<https://github.com/SocialfiPanda/privacy-cash-core-evm/pull/6>

PCM-3: Violates the Checks-Effects-Interactions Pattern in **transact**

Type: Informational

Files affected:

- contracts/EtherPool.sol

Description: In **transact** function, **_insert** function is called after External calls (**_extData.recipient.call...**) which updates multiple state variables. This violates the Checks-Effects-Interactions Pattern for reentrancy vulnerability. It is recommended to follow Checks-Effects-Interactions Pattern hence any updates should be done before doing any external call. It should be noted that the function uses a **nonReentrant** modifier hence prevents Same-function reentrancy attack.

Recommendation: call **_insert** function before doing an external call.

Status: The team acknowledged it but decided to go with the current approach as it is done in the same way in TornadoCash.

PCM-4: Custom implementation of **nonReentrant** modifier instead of using OpenZeppelin's reentrancy guard contract

Type: Informational

Files affected:

- contracts/EtherPool.sol

Description: The **transact** function uses a custom reentrancy guard by implementing **nonReentrant** modifier for preventing reentrancy attacks. While this functionality is found to be sound one can use the battle-tested OpenZeppelin version of reentrancy guard.

Recommendation: Recommended to use OpenZeppelin version of reentrancy guard.

Status: The team modified to use OpenZeppelin version of reentrancy guard in this PR: <https://github.com/SocialfiPanda/privacy-cash-core-evm/pull/6>

PCM-5: Floating Solidity pragma directives are used

Type: Informational

Files affected:

- contracts/EtherPool.sol

Description: Multiple files use `pragma solidity ^0.8.0;` version. This allows compilation with any 0.8.x compiler. Different compiler versions may produce different bytecodes and may have subtle behavior differences. Therefore it is recommended to pin to a specific tested version.

Recommendation: Pin the pragma version to a specific tested version.

Status: The team pinned to a specific version here:

<https://github.com/SocialfiPanda/privacy-cash-core-evm/pull/5>

PCM-6: Missing check to ensure that `minimumAmount` is less than `maximumDepositAmount`

Type: low

Files affected:

- contracts/EtherPool.sol

Description: In the PR #8, minimum deposit amount was added to prevent dust deposit griefing however, neither `_configureMinimumAmount` nor `_configureMaximumDepositAmount` validates that `minimumAmount` $\leq$ `maximumDepositAmount`. If an admin sets or updates `minimumAmount` $>$ `maximumDepositAmount`, all deposits and withdrawals will be permanently blocked until reconfigured. Therefore, it is recommended to add a check ensuring `minimumAmount` is less or equal to `maximumDepositAmount`.

Recommendation: Add check ensuring `minimumAmount` $\leq$ `maximumDepositAmount` in both functions `_configureMinimumAmount` and `_configureMaximumDepositAmount`

Status: The team add the check in the commit:

<https://github.com/SocialfiPanda/privacy-cash-core-evm/pull/8/changes/38fd016bf6e92a7bb77c88e32ca6de7c8204747f>

References

- [Slither](#)
- [Tornado Nova](#)
- [Privacy Cash](#)

Appendix

Results from Slither Static Analysis tools(Clipped)

```
slither contracts --solc-remaps @openzeppelin/=node_modules/@openzeppelin/
--exclude-dependencies
'solc --version' running
'solc @openzeppelin/=node_modules/@openzeppelin/ contracts/EtherPool.sol --combined-json
abi,ast,bin,bin-runtime,srcmap,srcmap-runtime,userdoc,devdoc,hashes --allow-paths
./Users/manishspc/HashCloak/audit/privacyCash/privacy-cash-core-evm/contracts' running
'solc --version' running
'solc @openzeppelin/=node_modules/@openzeppelin/ contracts/Proxy.sol --combined-json
abi,ast,bin,bin-runtime,srcmap,srcmap-runtime,userdoc,devdoc,hashes --allow-paths
./Users/manishspc/HashCloak/audit/privacyCash/privacy-cash-core-evm/contracts' running
'solc --version' running
'solc @openzeppelin/=node_modules/@openzeppelin/ contracts/MerkleTreeWithHistory.sol
--combined-json abi,ast,bin,bin-runtime,srcmap,srcmap-runtime,userdoc,devdoc,hashes
--allow-paths
./Users/manishspc/HashCloak/audit/privacyCash/privacy-cash-core-evm/contracts' running
'solc --version' running
'solc @openzeppelin/=node_modules/@openzeppelin/ contracts/Verifier2.sol --combined-json
abi,ast,bin,bin-runtime,srcmap,srcmap-runtime,userdoc,devdoc,hashes --allow-paths
./Users/manishspc/HashCloak/audit/privacyCash/privacy-cash-core-evm/contracts' running
INFO:Detectors:
EtherPool.transact(EtherPool.Proof,EtherPool.ExtData) (contracts/EtherPool.sol#88-120) sends
eth to arbitrary user
    Dangerous calls:
        - (success,None) = _extData.recipient.call{value: uint256(- _extData.extAmount)}{()}
(contracts/EtherPool.sol#105)
        - (success_scope_0,None) = _extData.feeRecipient.call{value: _extData.fee}{()}
(contracts/EtherPool.sol#111)
Reference:
https://github.com/crytic/slither/wiki/Detector-Documentation#functions-that-send-ether-to-a
rbitrary-destinations
INFO:Detectors:
```

```

Verifier2.verifyProof(uint256[2],uint256[2][2],uint256[2],uint256[7])
(contracts/Verifier2.sol#77-209) calls Verifier2.verifyProof.asm_0.checkField()
(contracts/Verifier2.sol#79-84) which halt the execution return(uint256,uint256)(0,0x20)
(contracts/Verifier2.sol#82)
Verifier2.verifyProof(uint256[2],uint256[2][2],uint256[2],uint256[7])
(contracts/Verifier2.sol#77-209) calls Verifier2.verifyProof.asm_0.checkPairing()
(contracts/Verifier2.sol#112-181) which halt the execution return(uint256,uint256)(0,0x20)
(contracts/Verifier2.sol#108)
Verifier2.verifyProof.asm_0.checkPairing() (contracts/Verifier2.sol#112-181) calls
Verifier2.verifyProof.asm_0.g1_mulAccC() (contracts/Verifier2.sol#87-110) which halt the
execution return(uint256,uint256)(0,0x20) (contracts/Verifier2.sol#98)
Reference:
https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-return-in-assembly
INFO:Detectors:
Reentrancy in EtherPool.transact(EtherPool.Proof,EtherPool.ExtData)
(contracts/EtherPool.sol#88-120):
  External calls:
    - (success,None) = _extData.recipient.call{value: uint256(- _extData.extAmount)}{ }
(contracts/EtherPool.sol#105)
    - (success_scope_0,None) = _extData.feeRecipient.call{value: _extData.fee}{ }
(contracts/EtherPool.sol#111)
  State variables written after the call(s):
    - _insert(_args.outputCommitments[0],_args.outputCommitments[1])
(contracts/EtherPool.sol#115)
      - currentRootIndex = newRootIndex (contracts/MerkleTreeWithHistory.sol#78)
      MerkleTreeWithHistory.currentRootIndex (contracts/MerkleTreeWithHistory.sol#25) can
be used in cross function reentrancies:
      - MerkleTreeWithHistory.currentRootIndex (contracts/MerkleTreeWithHistory.sol#25)
      - MerkleTreeWithHistory.getLastRoot() (contracts/MerkleTreeWithHistory.sol#108-110)
      - MerkleTreeWithHistory.isKnownRoot(bytes32)
(contracts/MerkleTreeWithHistory.sol#87-103)
    - _insert(_args.outputCommitments[0],_args.outputCommitments[1])
(contracts/EtherPool.sol#115)
      - roots[newRootIndex] = currentLevelHash
(contracts/MerkleTreeWithHistory.sol#79)
      MerkleTreeWithHistory.roots (contracts/MerkleTreeWithHistory.sol#23) can be used in
cross function reentrancies:
      - MerkleTreeWithHistory._initialize() (contracts/MerkleTreeWithHistory.sol#35-41)
      - MerkleTreeWithHistory.getLastRoot() (contracts/MerkleTreeWithHistory.sol#108-110)
      - MerkleTreeWithHistory.isKnownRoot(bytes32)
(contracts/MerkleTreeWithHistory.sol#87-103)
      - MerkleTreeWithHistory.roots (contracts/MerkleTreeWithHistory.sol#23)
Reference:
https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities
INFO:Detectors:
ERC1967Utils.upgradeToAndCall(address,bytes)
(node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#67-76) ignores return
value by Address.functionDelegateCall(newImplementation,data)
(node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#72)
ERC1967Utils.upgradeBeaconToAndCall(address,bytes)
(node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#157-166) ignores return

```

```

value by Address.functionDelegateCall(IBeacon(newBeacon).implementation(),data)
(node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#162)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unused-return
INFO:Detectors:
Reentrancy in EtherPool.transact(EtherPool.Proof,EtherPool.ExtData)
(contracts/EtherPool.sol#88-120):
    External calls:
        - (success,None) = _extData.recipient.call{value: uint256(- _extData.extAmount)}{()}
(contracts/EtherPool.sol#105)
        - (success_scope_0,None) = _extData.feeRecipient.call{value: _extData.fee}{()}
(contracts/EtherPool.sol#111)
    State variables written after the call(s):
        - _insert(_args.outputCommitments[0],_args.outputCommitments[1])
(contracts/EtherPool.sol#115)
            - filledSubtrees[i] = currentLevelHash
(contracts/MerkleTreeWithHistory.sol#68)
            - _insert(_args.outputCommitments[0],_args.outputCommitments[1])
(contracts/EtherPool.sol#115)
            - nextIndex = _nextIndex + 2 (contracts/MerkleTreeWithHistory.sol#80)
Reference:
https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-2
INFO:Detectors:
7 different versions of Solidity are used:
    - Version constraint ^0.8.0 is used by:
        - ^0.8.0 (contracts/EtherPool.sol#3)
        - ^0.8.0 (contracts/MerkleTreeWithHistory.sol#3)
    - Version constraint >=0.7.0<0.9.0 is used by:
        - ->=0.7.0<0.9.0 (contracts/Verifier2.sol#21)
    - Version constraint >=0.4.11 is used by:
        - ->=0.4.11 (node_modules/@openzeppelin/contracts/interfaces/IERC1967.sol#4)
    - Version constraint >=0.4.16 is used by:
        - ->=0.4.16
(node_modules/@openzeppelin/contracts/interfaces/draft-IERC1822.sol#4)
        - ->=0.4.16 (node_modules/@openzeppelin/contracts/proxy/beacon/IBeacon.sol#4)
    - Version constraint ^0.8.21 is used by:
        - ^0.8.21
(node_modules/@openzeppelin/contracts/proxy/ERC1967/ERC1967Utils.sol#4)
    - Version constraint ^0.8.20 is used by:
        - ^0.8.20
(node_modules/@openzeppelin/contracts/proxy/utils/Initializable.sol#4)
        - ^0.8.20 (node_modules/@openzeppelin/contracts/utils/Address.sol#4)
        - ^0.8.20 (node_modules/@openzeppelin/contracts/utils/Errors.sol#4)
        - ^0.8.20 (node_modules/@openzeppelin/contracts/utils/LowLevelCall.sol#4)
        - ^0.8.20 (node_modules/@openzeppelin/contracts/utils/StorageSlot.sol#5)
    - Version constraint ^0.8.22 is used by:
        - ^0.8.22
(node_modules/@openzeppelin/contracts/proxy/utils/UUPSUpgradeable.sol#4)
Reference:
https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used
INFO:Detectors:

```

Version constraint ^0.8.0 contains known severe issues

(<https://solidity.readthedocs.io/en/latest/bugs.html>)

- FullInlinerNonExpressionSplitArgumentEvaluationOrder
- MissingSideEffectsOnSelectorAccess
- AbiReencodingHeadOverflowWithStaticArrayCleanup
- DirtyByteArrayToStorage
- DataLocationChangeInInternalOverride
- NestedCalldataArrayAbiReencodingSizeValidation
- SignedImmutables
- ABIDecodeTwoDimensionalArrayMemory
- KeccakCaching.

It is used by:

- ^0.8.0 (contracts/MerkleTreeWithHistory.sol#3)

Version constraint ^0.8.20 contains known severe issues

(<https://solidity.readthedocs.io/en/latest/bugs.html>)

- VerbatimInvalidDeduplication
- FullInlinerNonExpressionSplitArgumentEvaluationOrder
- MissingSideEffectsOnSelectorAccess.

It is used by:

- ^0.8.20 (node_modules/@openzeppelin/contracts/proxy/utils/Initializable.sol#4)

Reference:

<https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity>